

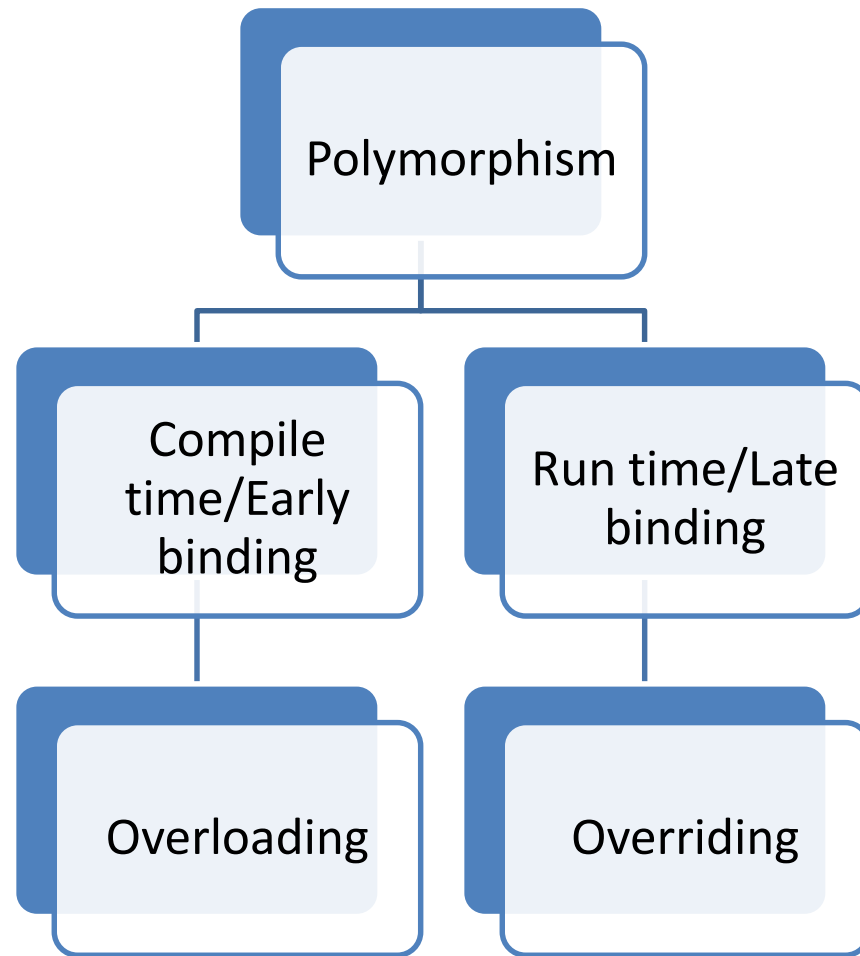
Object-oriented Programming

Week 8 | Lecture 1

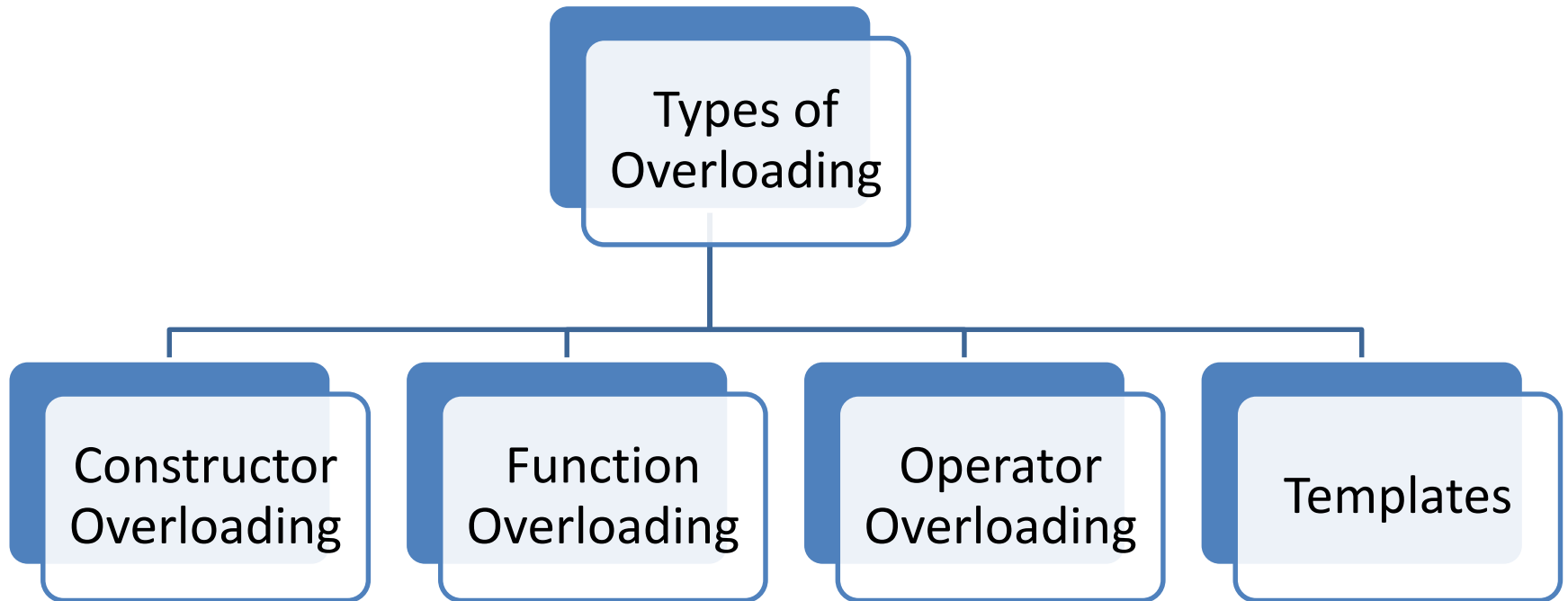
Polymorphism

- Polymorphism refers to the ability of a method to be used in different ways, that is, it can take different forms at different times (poly + morphos).
- Polymorphism enables us to “program in the general” rather than “program in the specific.”

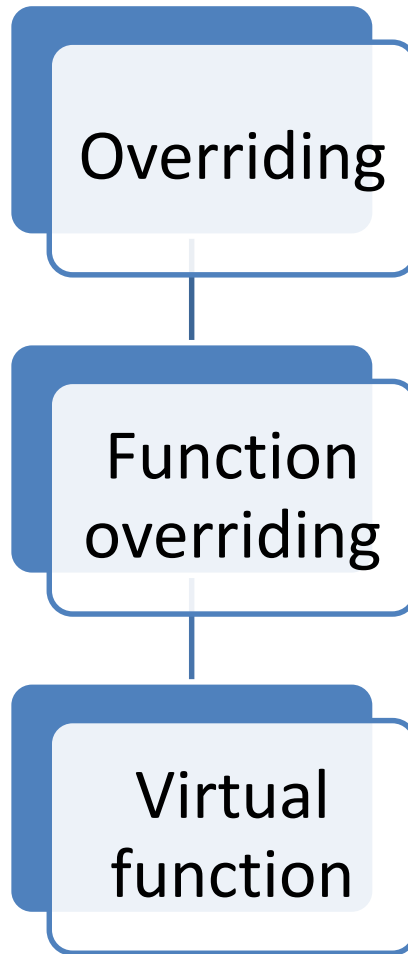
Types of polymorphism:



Compile time / Static / Early Binding polymorphism



Run time / Dynamic / Late Binding polymorphism



- static : memory allocated at compile time
- In this case, during compile time when the binding happens, compiler links the method definition with the method call. So it gets resolved during compile time
- dynamic : memory allocated at run time

Overloading in C++

- Whenever same method name is existing multiple times in the **same class** with **different number of parameter** or **different order of parameters** or **different types of parameters** is known as method overloading.

Function Overloading

- C++ enables several functions of the same name to be defined, as long as they have different parameter signatures
- The compiler selects the proper function to call by examining the number, types and order of the arguments in the call

Function Overloading

- Function overloading allows the program to follow the principle of “*Polymorphism*”
- The *return type* or *name* of parameters don't help the compiler in selecting which overloaded function to call
- Overloading the functions that perform closely related tasks can make programs more readable and understandable

Example

void findPerson(string name) { . . . }

void findPerson(int ID) { . . . }

void findPerson(int ID, string addr) { . . . }

void findPerson(string addr, int ID) { . . . }

All of above are valid overloaded functions

Example

```
void showVal(int a)  
{ cout << a; }
```

```
void showVal(char a)  
{ cout << a; }
```

```
void showVal(float a)  
{ cout << a; }
```

```
int main()  
{  
    showVal('M');  
    showVal(20);  
    showVal(3.5);  
}
```

Can we overload a function on const and non const keyword?

- *A const member function can be overloaded with a non-const version. The compiler chooses which overloaded member function to use based on the object on which the function is invoked. If the object is const, the compiler uses the const version. If the object is not const, the compiler uses the non-const version.*
- *Note: Invoking a non-const member function on a const object is a compilation error*

- #include<iostream>
- using namespace std;
-
- class Test
- {
- protected:
- int x;
- public:
- Test (int i):x(i) { }
- void fun() const
- {
- cout << "fun() const called " << endl;
- }
- void fun()
- {
- cout << "fun() called " << endl;
- }
- };
- int main()
- {
- Test t1 (10);
- const Test t2 (20);
- t1.fun();
- t2.fun();
- return 0;
- }

```
fun() called  
fun() const called
```

The two methods 'void fun() const' and 'void fun()' have same signature except that one is const and other is not. Also, if we take a closer look at the output, we observe that, 'const void fun()' is called on const object and 'void fun()' is called on non-const object.

Pitfall

- Avoid using default arguments with overloaded functions as it can be dangerous i.e. can lead to error conditions that are hard to trace

For example:

- In a program, a function that takes no argument and another function of the same name that takes all default arguments

- `#include <iostream>`
- `using namespace std;`
- `class printData {`
- `public:`
- `void print() {`
- `cout << "Printing default " << endl;}`
- `void print(float f) {`
- `cout << "Printing float: " << f << endl; };`
- `int main(void) {`
- `printData pd;`
- `pd.print(); // Call print to print default`
- `pd.print(5.6); // Call print to print float`
- `return 0;}`

 C:\Users\Nida Munawar\Documents\Untitled2.exe

Printing default

Printing float: 5.6

Process exited after 0.1825 seconds with return value 0

Press any key to continue . . .

- `#include <iostream>`
- `using namespace std;`
- `class printData {`
- `public:`
- `void print() {`
- `cout << "Printing default " << endl; }`
- `void print(float f=0) {`
- `cout << "Printing float: " << f << endl; };`
- `int main(void) {`
- `printData pd;`
- `pd.print(); //[Error] call of overloaded 'print()' is ambiguous`
- `pd.print(5.6);`
- `return 0;}`

Is Function Overloading supported in C?

- Any object code produced by compilers is usually linked with other pieces of object code by a type of program called a linker. The linker needs a great deal of information on each program entity. For example, to correctly link a function it needs its name, the number of arguments and their types, and so on. These requirements enable use of function overloading in C++. C, only distinguished the pieces of code by their name that's why function overloading is not supported by C

Name Mangling

- How does C++ compiler distinguishes between different functions when it generates object code – it changes names by adding information about arguments. This technique of adding additional information to function names is called Name Mangling.

Name Mangling

- The compiler encodes each function identifier with the number and types of its parameters
- This is called **name mangling** or **name decoration**

Name Mangling

- C++ standard doesn't specify any particular technique for name mangling, so different compilers may append different information to function names.

g++ name_decoration / Name Mangling

- Each mangled name (other than main) begins with two underscores (__) followed by the letter Z, a number and the function name
- The number that follows Z specifies how many characters are in the function's name
- The function name is then followed by an encoding of its parameter list

Name Mangling

```
void func(int i, char c) { . . . }
```

```
int func2(string x, float y) { . . . }
```

```
void func3(int a, float b, int& c) { . . . }
```

Mangled Names:

```
__4Zfuncic
```

```
__5Zfunc2sf
```

```
__5Zfunc3ifRi
```


Question

- You cannot overload function declarations that differ only by return type.

WHY?

- Return type of functions is not a part of the mangled name which is generated by the compiler for uniquely identifying each function in the case of function overloading.

No of arguments

Type of arguments &

Sequence of arguments

are the parameters which are used to generate the unique mangled name for each function. It is on the basis of these unique mangled names that compiler can understand which function to call even if the names are same.

Can we overload functions on const and non const parameters?

- `#include <iostream>`
- `using namespace std;`
- `class base{`
- `public:`
- `void func(int a){`
- `cout << "inside base"<< endl; }`
- `void func(const int a){`
- `cout << "inside base"<< endl; };`
- `int main(){`
- `base b;`
- `b.func(3) ;}`

No!

Compiler (2) Resources Compile Log Debug Find Results Close			
Line	Col	File	Message
7	10	C:\Users\Nida\Documents\Untitled1.cpp	[Error] 'void base::func(int)' cannot be overloaded
5	9	C:\Users\Nida\Documents\Untitled1.cpp	[Error] with 'void base::func(int)'

- C++ Overload Static Function with Non-Static Function ??????????
- **Void print(){}**
- **Static void print (){}**

No, it is directly prohibited by the standard:

ISO 14882:2003 C++ Standard 13.1/2 – Overloadable declarations

Certain function declarations cannot be overloaded:

- Function declarations that differ only in the return type cannot be overloaded.
- Member function declarations with the same name and the same parameter types
- cannot be overloaded if any of them is a static member function declaration (9.4).

Question

- Can we overload main() function in C++?

- in c++ global main cannot be overloaded it will generate compile time error, in is because you cannot have multiple entry point for the same program
- since when you run the program the program will not know where to begin
- But if you write main function inside a class then it will compile fine however it will not be treated as program entry point

Exercise

- Create a **Message** class with a constructor that takes a single **string** with a default value. Create a private member **string**, and in the constructor simply assign the argument **string** to your internal **string**. Create two overloaded member functions called **print()**: one that takes no arguments and simply prints the message stored in the object, and one that takes a **string** argument, which it prints in addition to the internal message. Also write name mangling for these overloaded methods

Next Lecture

- Operator Overloading!